

GradFix Work Orders - Design Gist

Main conclusion

Work Orders should be a separate operational subsystem, not just a PDF field on reports.

A report is the citizen-facing issue record.

A work order is an operational instruction sent to one responsible entity to perform work.

Recommended domain rule

One report can generate zero, one, or many work orders.

Why this matters

Real city problems often involve multiple services. For example, one traffic issue may need road repair, temporary signs, and later repainting. A single reports.assigned_entity_id field is not enough.

Core actors

- Citizen: creates report, receives updates, may confirm/rate resolution.
- Reviewer: accepts reports and verifies completed work.
- Conductor: creates, sends, retries, reassigns, cancels, and completes work orders.
- Tenant Admin: can override and manage all tenant work-order actions.
- Community Manager: can view/communicate, but should not normally create or cancel work orders.
- Responsible Entity: receives the work order and performs the work.
- System: generates PDFs, sends emails, records delivery failures, and logs events.

Recommended entities

- work_orders
- work_order_documents
- work_order_deliveries
- work_order_events
- optional work_order_photos
- optional work_order_items

Recommended relationships

- reports 1:N work_orders
- responsible_entities 1:N work_orders
- work_orders 1:N work_order_documents
- work_orders 1:N work_order_deliveries
- work_orders 1:N work_order_events
- report_photos N:M work_orders, if staff can choose which photos are included

Recommended work order states

- draft
- ready_to_send
- sending
- sent
- delivery_failed
- acknowledged
- in_progress
- completed
- verified
- cancelled
- superseded
- closed

Minimum MVP states

- draft
- sent
- delivery_failed
- in_progress
- completed
- cancelled
- superseded

Report lifecycle relationship

Report status and work order status should not be the same thing.

Report status is citizen-facing:

new -> accepted -> assigned -> in_progress -> resolved -> closed

Work order status is operational:

draft -> sent -> in_progress -> completed -> verified/closed

A report should become in_progress when at least one required work order is active.

A report should become resolved only when required work orders are completed or verified.

Important rules

- Work orders should usually be created manually by Conductor or Tenant Admin after a report is accepted.
- Automatic routing can suggest the responsible entity, but should not silently send a work order.
- Sent PDFs must be immutable snapshots.
- Regenerating a sent PDF should create a new document version.
- Reassigning after send should usually supersede/cancel the old work order and create a new one.
- Email failure must not lose the work order.
- Failed delivery should be visible and retryable.
- Closing a report while work is active should be blocked or require Tenant Admin override with reason.
- Duplicate merge should require special review if either report already has active/sent work orders.

Audit events required

- work_order.created
- work_order.updated
- work_order.status_changed
- work_order.assigned_entity_changed
- work_order.pdf_generated
- work_order.pdf_regenerated
- work_order.email_queued
- work_order.email_sent
- work_order.email_failed
- work_order.email_retried
- work_order.cancelled
- work_order.superseded
- work_order.completed
- work_order.verified
- work_order.reopened

Actions requiring a reason

- cancel work order
- reassign after send

- regenerate after send
- supersede work order
- reopen completed work
- close report while work is active
- merge duplicate reports when work orders exist

Email flow

1. Staff reviews work order.
2. PDF snapshot is generated.
3. Delivery record is created as queued.
4. Email job sends PDF or secure PDF link.
5. Delivery becomes sent or delivery_failed.
6. Failed delivery can be retried.
7. Every attempt is audited.

PDF flow

1. Snapshot report details, location, priority, category, selected photos, and responsible entity.
2. Generate immutable PDF document.
3. Store file key, checksum, generated_by, generated_at.
4. Sending uses the current PDF version.
5. Regeneration after send creates a new version.

Security requirements

- Every work order must have tenant_id.
- Every query must filter by tenant_id.
- Tenant-bound users must not access another tenant's work orders.
- PDFs should be private or use signed URLs.
- Reporter PII should not be included by default.
- Email templates must escape user-generated content.
- All send/regenerate/cancel actions must be audited.

Three architecture options

Option 1: Add PDF fields directly to reports

Pros: fastest, easiest demo.

Cons: cannot support multiple services, retries, document versions, real audit, or proper cancellation.

Verdict: not recommended.

Option 2: One work_order per report

Pros: cleaner than report fields, separate status.

Cons: still assumes one service per report and will need redesign later.

Verdict: better, but too limiting.

Option 3: Full Work Order subsystem

Pros: supports multiple services, multiple work orders, retries, immutable PDFs, audit, reassignment, and future contractor portal.

Cons: more design and implementation work.

Verdict: recommended.

Best recommendation

Use Option 3.

Build Work Orders as first-class operational records with their own lifecycle, documents, deliveries, and audit events. Keep the UI simple at first, but do not compromise the database/domain

model.

Minimum safe MVP

- reports 1:N work_orders
- one responsible entity per work order
- states: draft, sent, delivery_failed, in_progress, completed, cancelled, superseded
- immutable PDF document records
- delivery attempt records
- audit events for every important action
- Conductor/Tenant Admin can create/send/cancel
- Reviewer can verify completion

Final takeaway

Do not implement Work Orders as just "generate PDF from report". That would create technical debt and make real municipal workflows hard to support later.